Short Term Load Forecasting using Machine Learning

Made with GAMMA

Table of Contents

1. Introduction
2. Literature Review
3. Problem statement
4. Methodology
5. Dataset Information
6. Exploratory data analysis
7. Data Preprocessing
8. Machine Learning models
9. Evaluation Metrics
10. Results
11. Conclusion
12. References

Made with GAMMA

Introduction

- Short-Term Load Forecasting (STLF) predicts the electrical load for the next few hours up to 24 hours. It is essential for real-time power system operation and decision-making.
- STLF helps utilities schedule generation, manage energy flow, reduce operational costs, and maintain system reliability by anticipating demand fluctuations.
- Electrical load is highly influenced by temperature, time, season, and human activities, making STLF challenging and requiring accurate predictive models.
- Modern STLF uses machine learning and hybrid optimization models (like XGBoost, CatBoost with optimizers) to capture nonlinear patterns and improve forecasting accuracy.

Made with GAMMA

Problem Statement

- Electricity load changes every hour due to temperature, season, and human activities, making short-term forecasting difficult.
- Traditional methods (ARIMA, regression) cannot capture these nonlinear and fast-changing patterns accurately.
- Therefore, there is a need for machine learning models that can learn complex relationships and provide more accurate short-term load predictions.

Made with GAMMA

Methodology

1. Data understanding and preparation
2. Feature Engineering
3. Data Splitting and Scaling
4. Model Development
5. Model Evaluation
6. Comparison of results

Made with GAMMA

Dataset Information

- The dataset was obtained from Mendeley Data for short-term load forecasting analysis.
- It consists of hourly electricity load measurements with corresponding timestamps.
- Weather information in the form of temperature is included as an exogenous variable.
- The dataset spans a continuous time period, making it suitable for time-series modeling.
- All variables are numerical and continuous in nature.
- The dataset supports analysis of daily and weekly consumption patterns.

Made with GAMMA

Project timeline

Stage	Planned Activities
Stage 1	Dataset collection and exploration, data cleaning, handling missing values, feature engineering, and creation of a structured input feature matrix.
Stage 2	Implementation of the four proposed machine learning models: XGBoost Regressor, CatBoost Regressor, Support Vector Regression (SVR), and Random Forest Regressor, along with hyperparameter tuning and cross-validation.
Stage 3	Model evaluation using MAE, RMSE, and MAPE; comparative performance analysis and training-time assessment; and analysis of feature importance and model interpretability.
Stage 4	Result visualization, discussion of findings, report writing, proposal refinement, and preparation of final documentation and presentation slides.

Made with GAMMA

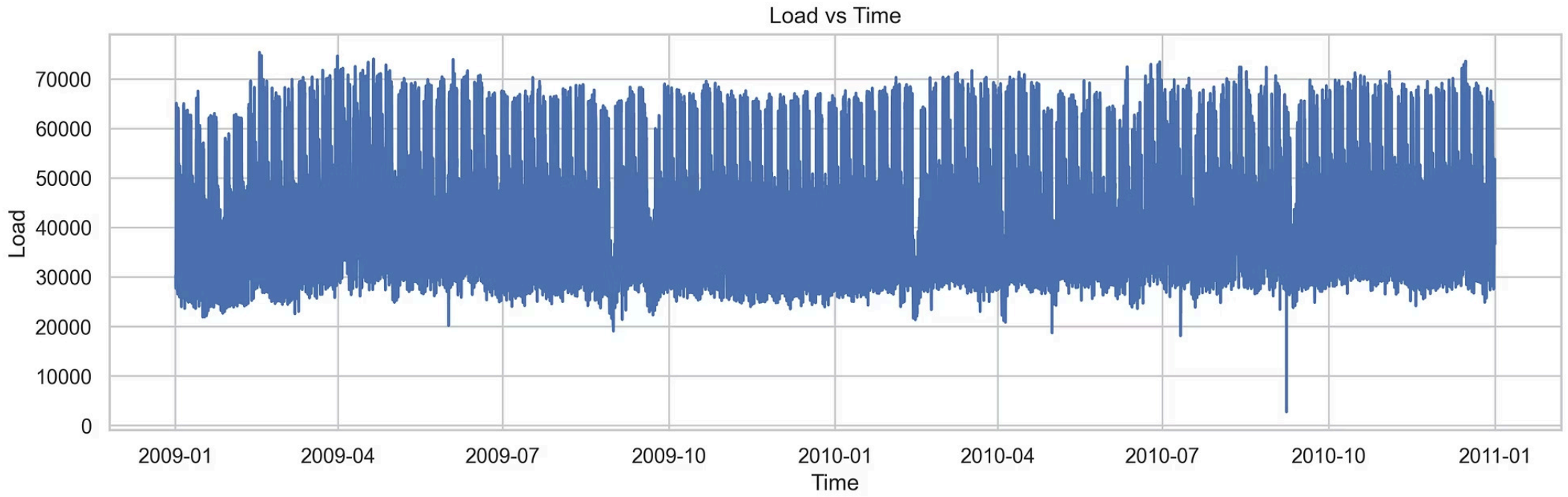
Exploratory Data Analysis

- EDA was performed on the data to understand the dependencies in the data
- Focused on trend, seasonality, variability, and correlations.
- Insights from EDA directly guided feature engineering decisions.
- No feature engineering or cleaning was applied before EDA.

Made with GAMMA

Load vs Time

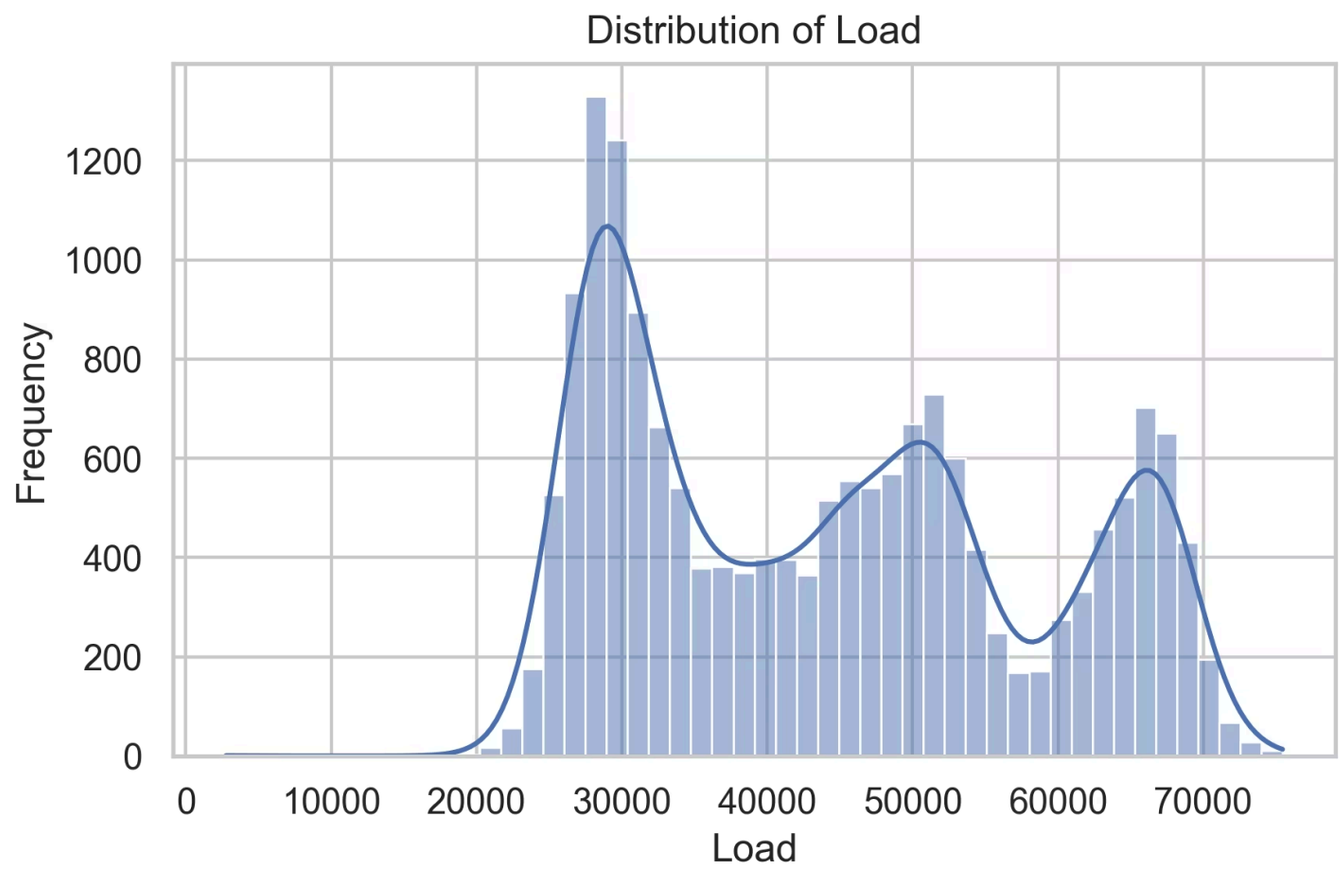
- Load vs time graph was plotted to observe overall trend and seasonality.
- Repeating daily and weekly patterns are visible in the load behavior.
- Sudden spikes and fluctuations in electricity demand can be identified.



Made with GAMMA

Load Distribution

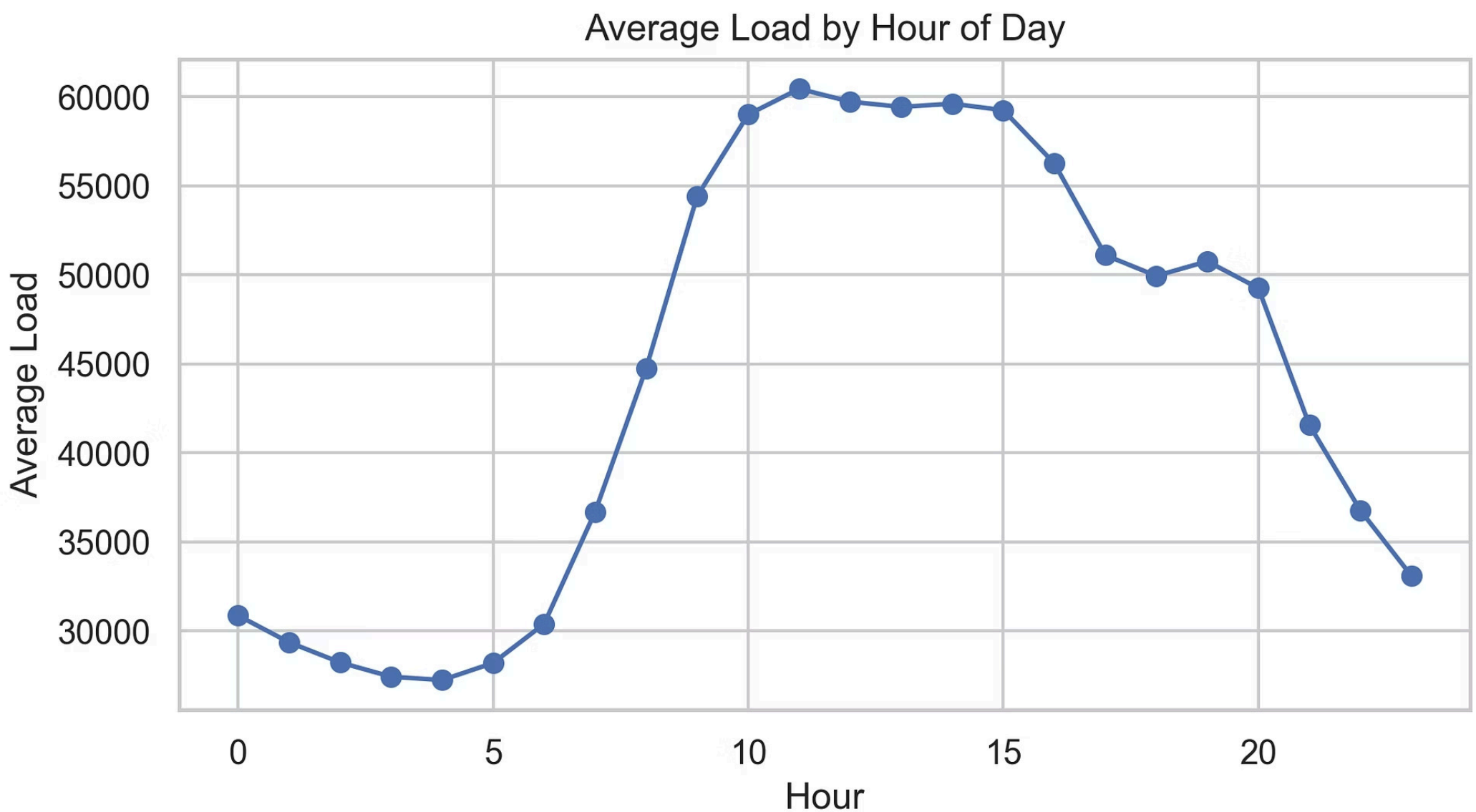
- Load distribution was plotted to understand statistical spread of demand values.
- Skewness and variability in electricity consumption can be observed.
- Presence of extreme load values can be identified.



Made with GAMMA

Average Load by Hour of Day

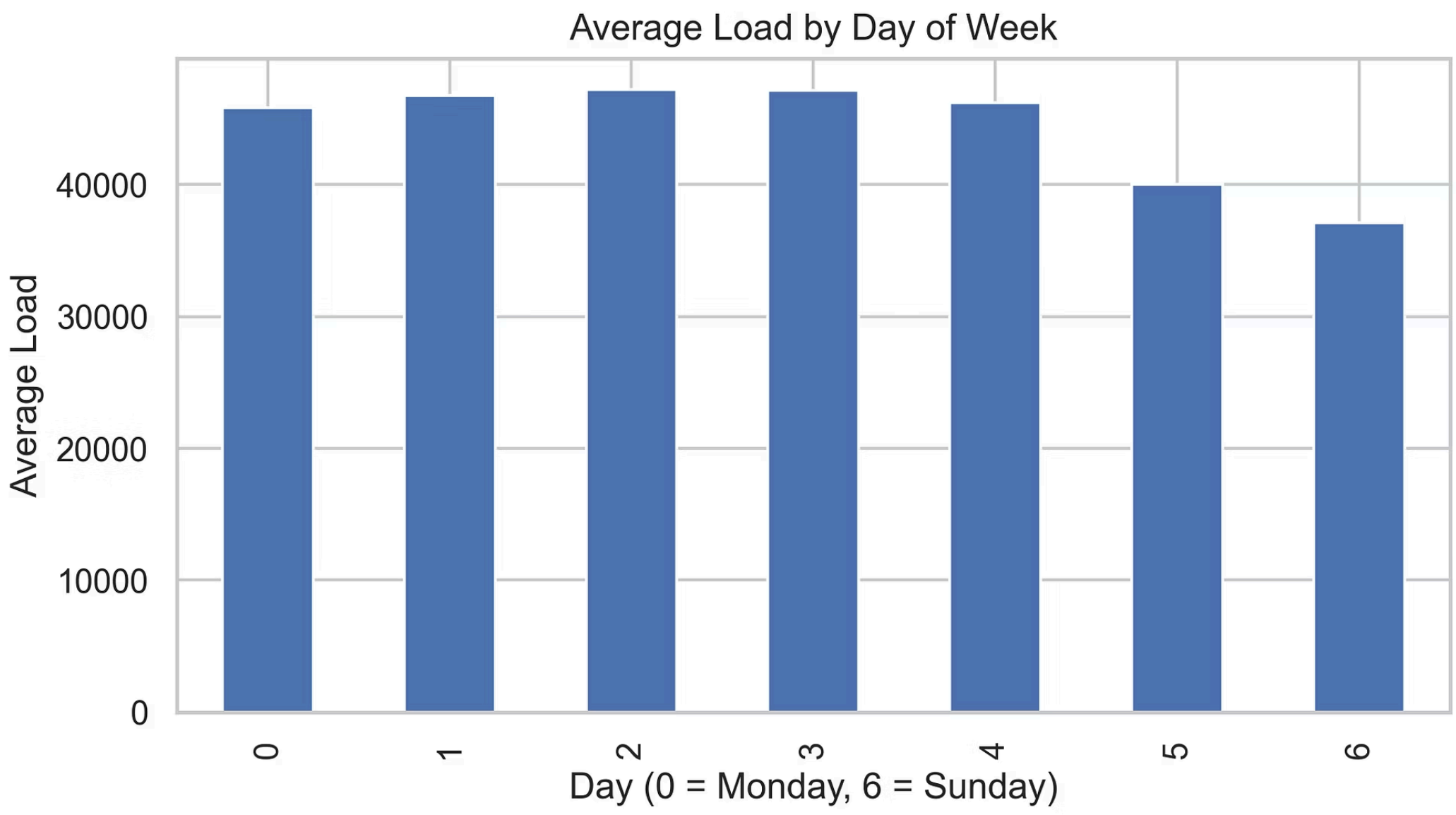
- Average load by hour graph was plotted to analyze intra-day consumption patterns.
- Peak and off-peak demand hours are clearly distinguishable.
- Hourly variation confirms dependency of load on time of day.



Made with GAMMA

Average Load by Day of Week

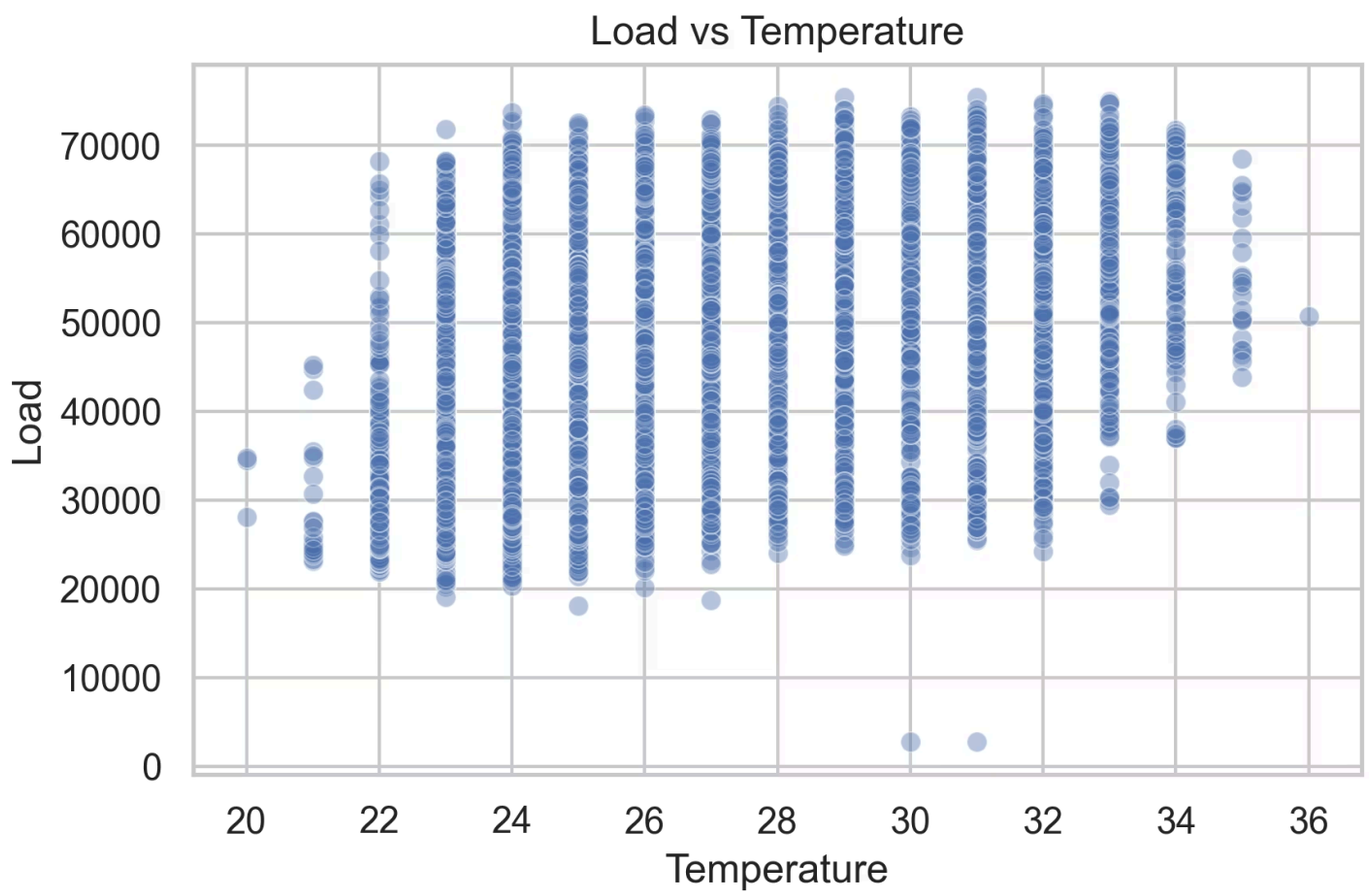
- Average load by day of week graph was plotted to study weekly seasonality.
- Differences between weekday and weekend consumption patterns are observed.
- Load demand shows systematic variation across days.



Made with GAMMA

Load vs Temperature

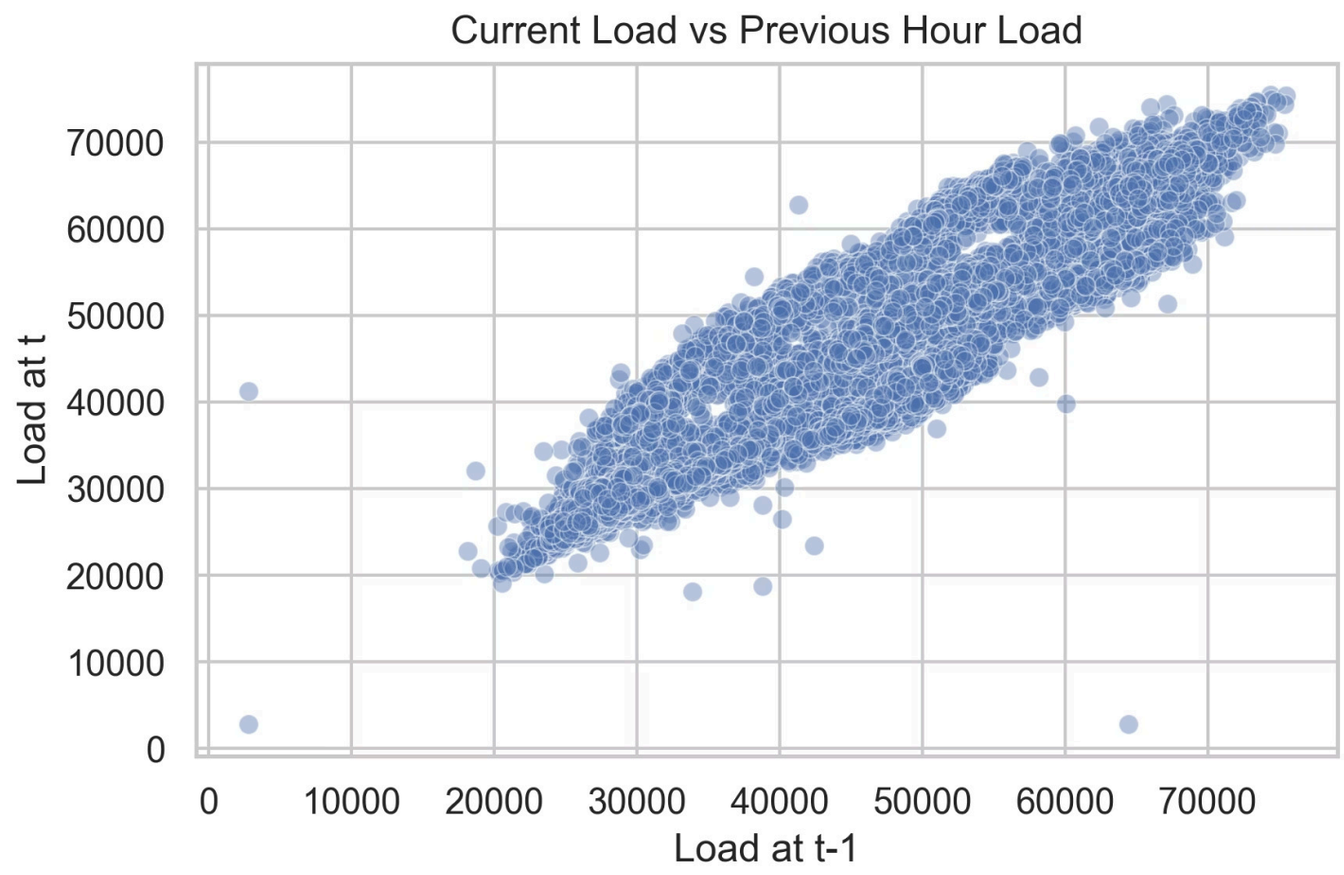
- Load vs temperature plot was generated to analyze weather dependency.
- Load demand increases with change in temperature.
- Temperature shows influence on electricity consumption.



Made with GAMMA

Current Load vs Previous Hour Load

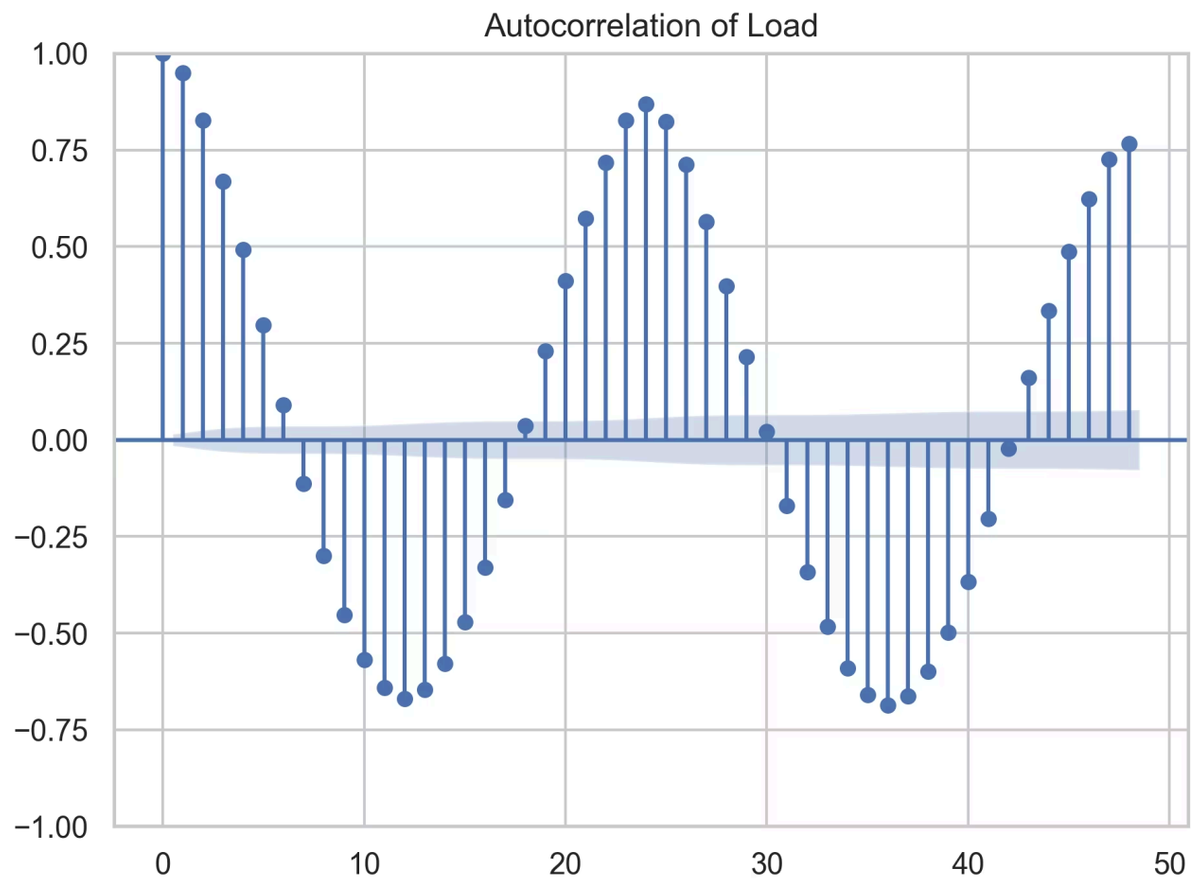
- Current load vs previous hour load plot was generated to examine short-term dependency.
- Strong correlation between consecutive load values is observed.
- Load demand is highly dependent on its immediate past values.



Made with GAMMA

Autocorrelation Function

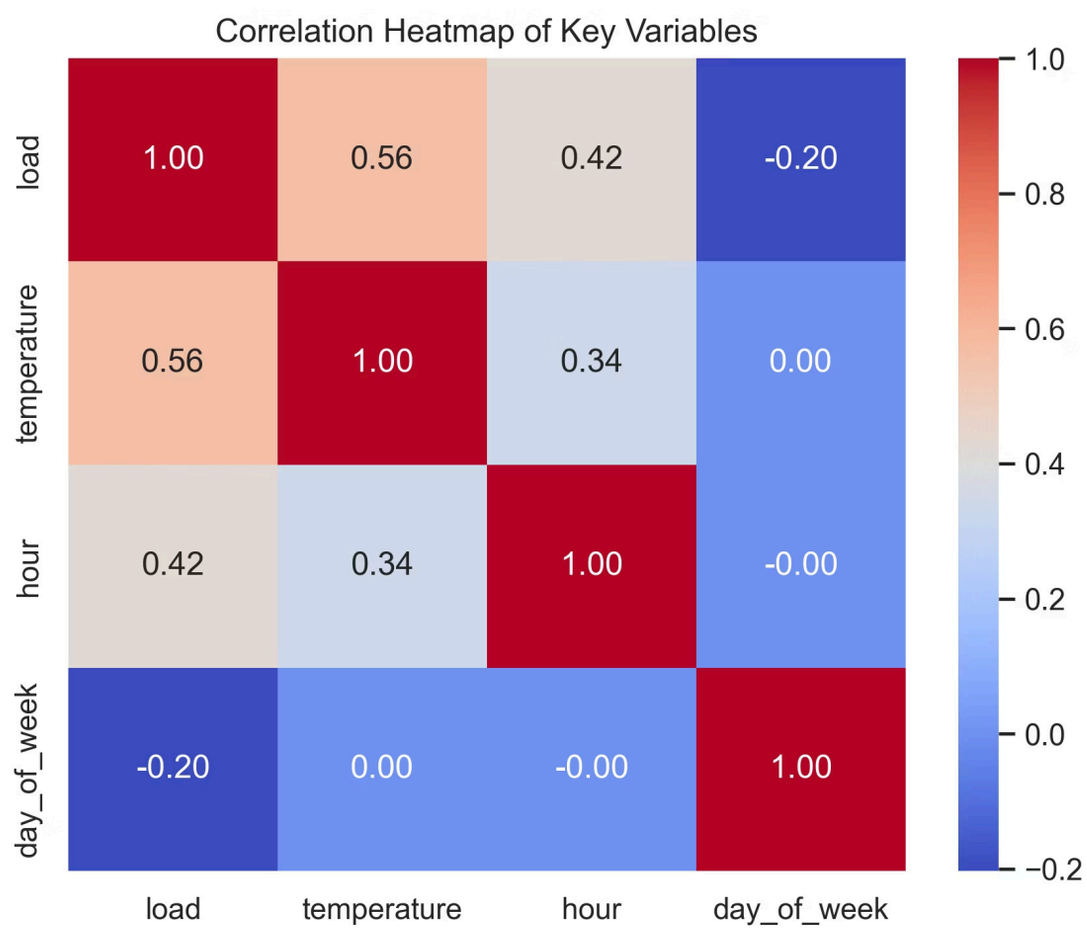
- Autocorrelation plot was generated to measure correlation across time lags.
- High autocorrelation is observed at short and daily lags.
- Temporal dependency in load data is clearly evident.



Made with GAMMA

Correlation Heatmap

- Correlation heatmap was generated to analyze relationships among key variables.



Made with GAMMA

Preprocessing

Preprocessing consists for data preparation, feature engineering, data splitting and scaling

We followed a structured 8-step preprocessing, because

- Raw hourly load dataset cannot be directly used for ML
- Classical ML models do not understand temporal dependencies
- We designed an 8-step preprocessing pipeline

Output: fully feature-engineered, leakage-free, model-ready data

Made with GAMMA

Step 1: Data Loading & Date-Time Processing

- Loaded raw CSV dataset
- Converted timestamp into `datetime` format
- Sorted data chronologically
- Ensured correct temporal order

Why Needed:

- Prevents data leakage
- Enables time-based feature extraction

Made with GAMMA

Step 2: Time-Based Feature Extraction

Extracted:

- Hour of day
- Day of month
- Month
- Day of week
- Weekend indicator

Why Needed:

- Load varies by time of day
- Weekday and weekend patterns differ
- Seasonal effects exist

Made with GAMMA

Step 3: Lag Feature Generation

Load lags:

- 1, 2, 3 hours
- 24, 48, 168 hours

Temperature lags:

- 1 hour
- 24 hours

Why Needed:

- ML models have no memory
- Lag features provide historical context
- Capture short-term and periodic dependencies

Made with GAMMA

Step 4: Rolling Statistical Features

Computed:

- Rolling mean (3, 6, 24 hours)
- Rolling standard deviation (24 hours)

Why Needed:

- Capture recent trends
- Smooth short-term fluctuations
- Measure demand volatility

Made with GAMMA

Step 5: Cyclical Encoding of Time Features

Applied sine & cosine encoding to:

- Hour of day
- Day of week

Why Needed:

- Time is cyclic in nature
- Prevents artificial discontinuities
- Improves model generalization

Made with GAMMA

Step 6: Data Cleaning

- Lag and rolling operations create missing values
- Removed rows containing NaNs
- Retained only complete samples

Why Needed:

- Avoids unreliable training data
- Prevents artificial data imputation

Made with GAMMA

Step 7: Time-Aware Train-Test Split

Dataset is split in this step

- 80% training data
- 20% testing data
- Chronological split only

Why Needed:

- Simulates real-world forecasting
- Prevents future data leakage

Made with GAMMA

Step 8: Feature Scaling (Model-Specific)

- Applied standardization only for SVR
- No scaling for tree-based models

Why Needed:

- SVR is distance-based
- Tree models are scale-invariant

Made with GAMMA

Machine Learning models being used

- CatBoost: An advanced boosting algorithm designed to reduce prediction bias and efficiently handle categorical and time-based features in forecasting tasks.
- XGBoost: A gradient boosting ensemble model that builds optimized decision trees sequentially to capture complex nonlinear relationships in load demand data.
- Random Forest Regressor: A bagging-based ensemble of decision trees that improves forecasting stability and accuracy by reducing variance through random feature selection.
- Support Vector Regression (SVR): A kernel-based regression technique that models nonlinear relationships by mapping input features into a high-dimensional space.

Made with GAMMA

SVR

Support Vector Regression (SVR) is a machine learning algorithm used for regression problems, which tries to find a function that predicts continuous values while keeping the prediction error within a certain tolerance. Instead of minimizing the total error like many regression algorithms, SVR tries to fit the best line or curve such that most of the data points fall within a predefined margin of tolerance.

Features

1. Margin-Based Regression: SVR introduces a margin called ε (epsilon), which defines an acceptable error range. If predictions fall within this margin, the error is not penalized.
2. Support Vectors: Only a few data points, called support vectors, influence the final model. These are the points outside or near the margin boundary. Other points inside the margin do not affect the model significantly.
3. Kernel Trick: SVR can model non-linear relationships using kernels. Common kernels include Linear kernel, Polynomial kernel, Radial Basis Function (RBF). The Kernel Trick allows SVR to transform data into a higher-dimensional space, making complex patterns easier to learn.
4. Regularization Control: SVR uses a parameter C (regularization parameter) to balance model complexity, prediction accuracy. If C is large then it tries to fit training data closely, if C is small it allows more flexibility and smoother models.
5. Works Well for High-Dimensional Data: SVR performs well when the dataset has many features, complex relationships. This makes it useful in fields like time series prediction and financial forecasting.

Made with GAMMA

Training Process in SVR

1. Input Dataset
2. Choose a Kernel Function
3. Define the Epsilon Margin
4. Identify Support Vectors
5. Optimize the Objective Function
6. Update the Regression Function
7. Apply Kernel Transformation
8. Final Regression Function
9. Prediction on New Data

Made with GAMMA

Random Forest

Random Forest is an ensemble machine learning algorithm that builds multiple decision trees and combines their predictions to produce a more accurate and stable result. A single decision tree can easily overfit the training data, meaning it learns the training data too closely and performs poorly on new data. Random Forest solves this problem by creating many decision trees and averaging their predictions.

Features

1. Ensemble Learning Method: Random Forest is an ensemble learning technique, meaning it combines multiple models to create a stronger and more reliable model. Instead of relying on a single decision tree, it uses many trees working together.
2. Reduces Overfitting: Single decision trees tend to overfit the data, but Random Forest reduces this by training trees on different subsets of data, using different feature subsets. This improves generalization on unseen data.
3. Bootstrap Sampling (Bagging): Random Forest uses a technique called Bootstrap Aggregating (Bagging). This means each tree is trained on a random sample of the dataset with replacement.
4. Random Feature Selection: At every split in the tree, Random Forest selects a random subset of features instead of considering all features. This ensures that trees become diverse and less correlated.
5. Robust and Stable Predictions: Because predictions come from many trees, Random Forest is less sensitive to noise, more stable, more accurate than a single tree

Made with GAMMA

Training process in Random Forest

1. Input Dataset
2. Create Multiple Bootstrap Samples
3. Train a Decision Tree for Each Sample
4. Grow the Trees Fully
5. Repeat for Many Trees
6. Prediction from Each Tree
7. Combine Predictions (Aggregation)
8. Final Model
9. Prediction on New Data

Made with GAMMA

XGBoost

XGBoost (Extreme Gradient Boosting) is an optimized and highly efficient implementation of the Gradient Boosting algorithm designed for high performance, scalability, and accuracy in machine learning tasks such as regression and classification. XGBoost belongs to the boosting family of ensemble learning algorithms, where multiple weak models (usually decision trees) are combined to form a strong predictive model.

Features

1. Gradient Boosting Framework: XGBoost is based on the Gradient Boosting technique, where models are trained sequentially to minimize prediction errors using gradient descent. Each new tree focuses on correcting the mistakes made by the previous trees.
2. Regularization to Prevent Overfitting: Unlike traditional boosting algorithms, XGBoost includes L1 and L2 regularization. This helps control model complexity, reduce overfitting, improve generalization on unseen data
3. Parallel Processing: XGBoost is designed to use parallel computing, which makes training much faster compared to traditional gradient boosting algorithms. This is especially useful for large datasets.
4. Handling Missing Values Automatically: XGBoost can automatically learn the best direction to handle missing values during tree construction. This means less preprocessing is required.
5. Tree Pruning: XGBoost uses depth-wise tree pruning based on a loss reduction threshold, which prevents unnecessary splits and improves efficiency.
6. High Performance and Scalability: XGBoost is optimized for speed, memory efficiency, distributed computing. This makes it suitable for large-scale machine learning problems.

Made with GAMMA

Training process in XGBoost

1. Input Dataset
2. Initialize the Base Prediction
3. Compute the Loss Function
4. Calculate Gradients (Residual Errors)
5. Train the First Decision Tree
6. Compute Tree Structure Using Gain
7. Apply Regularization
8. Update Predictions
9. Repeat the Boosting Process
10. Tree Pruning
11. Final Model
12. Prediction on New Data

Made with GAMMA

CatBoost

CatBoost (Categorical Boosting) is a gradient boosting algorithm designed to handle categorical features efficiently and reduce prediction bias. CatBoost is optimized to work with categorical data without heavy preprocessing. In traditional machine learning models, categorical features usually need one-hot encoding or label encoding, which can increase dimensionality and reduce performance. CatBoost solves this problem using ordered target statistics, allowing the model to learn from categorical variables directly.

Features

1. Native Handling of Categorical Features: CatBoost can directly process categorical variables without manual encoding. It converts them into meaningful numerical values using target statistics, improving model accuracy.
2. Ordered Boosting (Reduces Overfitting): Traditional gradient boosting sometimes introduces prediction shift (target leakage). CatBoost solves this using Ordered Boosting, where the model only uses past data when computing statistics, preventing the model from seeing future information.
3. Symmetric Decision Trees: CatBoost builds symmetric (balanced) decision trees. All nodes at the same depth use the same splitting condition, trees become faster to train and evaluate.
4. Reduced Overfitting: CatBoost uses Ordered boosting, Random permutations, Regularization so it reduces overfitting
5. Minimal Data Preprocessing: Unlike many algorithms, CatBoost handles missing values automatically, handles categorical variables automatically, requires very little feature engineering

Made with GAMMA

Training Process in CatBoost

1. Input Dataset
2. Handling Categorical Features
3. Random Permutation of Data
4. Initialize Prediction
5. Calculate Residual Errors
6. Train First Symmetric Decision Tree
7. Update Predictions
8. Repeat Boosting Process
9. Regularization and Overfitting Control
10. Final Model
11. Prediction on New Data

Made with GAMMA

Evaluation Metrics

1. **RMSE (Root Mean Square Error)**
   Measures the square root of the average squared difference between predicted and actual values.
   It penalizes large errors more strongly because the errors are squared.

Formula

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

where

y_i = actual value

$\hat{y}_i$ = predicted value

n = number of observations

2. **MAE (Mean Absolute Error)**
   Measures the average absolute difference between predicted and actual values.
   It treats all errors equally without emphasizing large errors.

Formula

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Made with GAMMA

Evaluation Metrics

3. **MAPE (Mean Absolute Percentage Error)**

Measures the average percentage difference between predicted and actual values.

It expresses prediction error relative to the magnitude of actual values.

Formula

$$MAPE = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

4. **sMAPE (Symmetric Mean Absolute Percentage Error)**

A modified version of MAPE that reduces bias when actual values are close to zero.

It uses the average of actual and predicted values in the denominator.

Formula

$$sMAPE = \frac{100}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{\frac{|y_i| + |\hat{y}_i|}{2}}$$

Made with GAMMA

Evaluation Metrics

5. **R^2 (Coefficient of Determination)**

Measures how well the model explains the variance in the actual data.

Values range from 0 to 1, where 1 indicates perfect prediction.

Formula

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where

$\bar{y}$ = mean of actual values.

6. **MBE (Mean Bias Error)**

Measures the average bias in predictions, indicating whether the model tends to overestimate or underestimate.

Positive MBE means Underprediction, negative MBE means Overprediction.

Formula

$$MBE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

Made with GAMMA

Results

Model	RMSE	MAE	MAPE (%)	sMAPE (%)	R^2	MBE
Random Forest	2019	1093	3.02	2.43	0.979	+413
SVR	4212	3048	7.77	6.79	0.909	+530
XGBoost	1927	1205	3.23	2.73	0.981	+334
CatBoost	2262	1448	4.04	3.35	0.974	+284

Made with GAMMA

Random Forest

RMSE: 2019

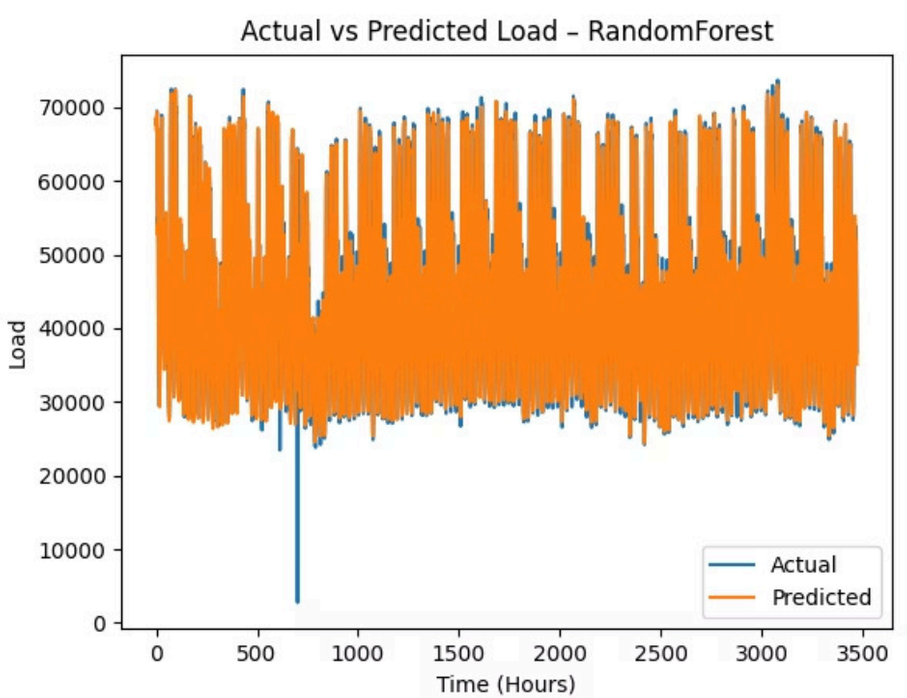
MAE: 1093

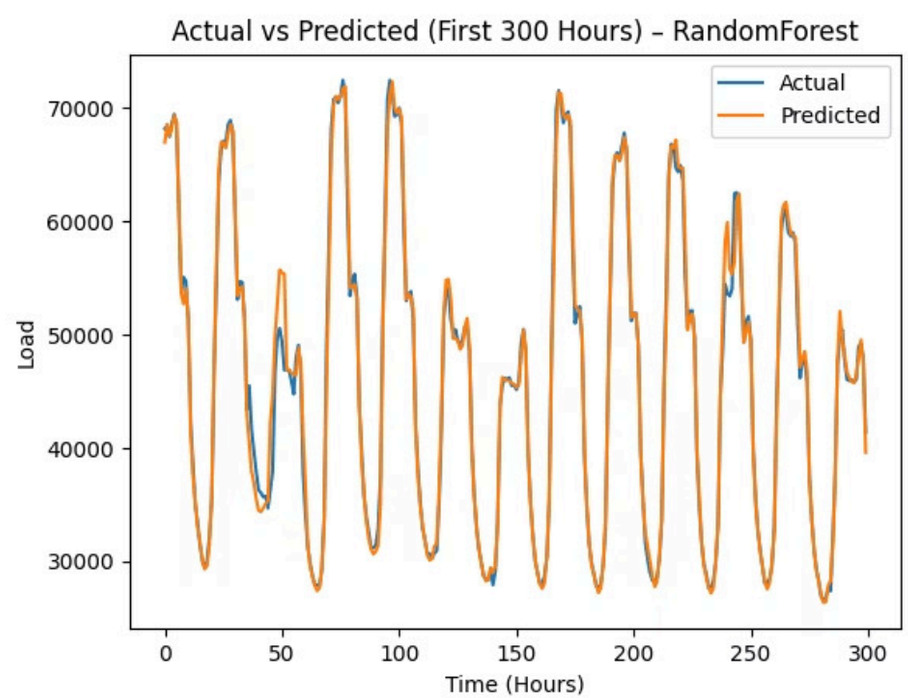
MAPE: 3.02%

sMAPE: 2.43%

R^2: 0.979

MBE: +413





Made with GAMMA

SVR

RMSE: 4212

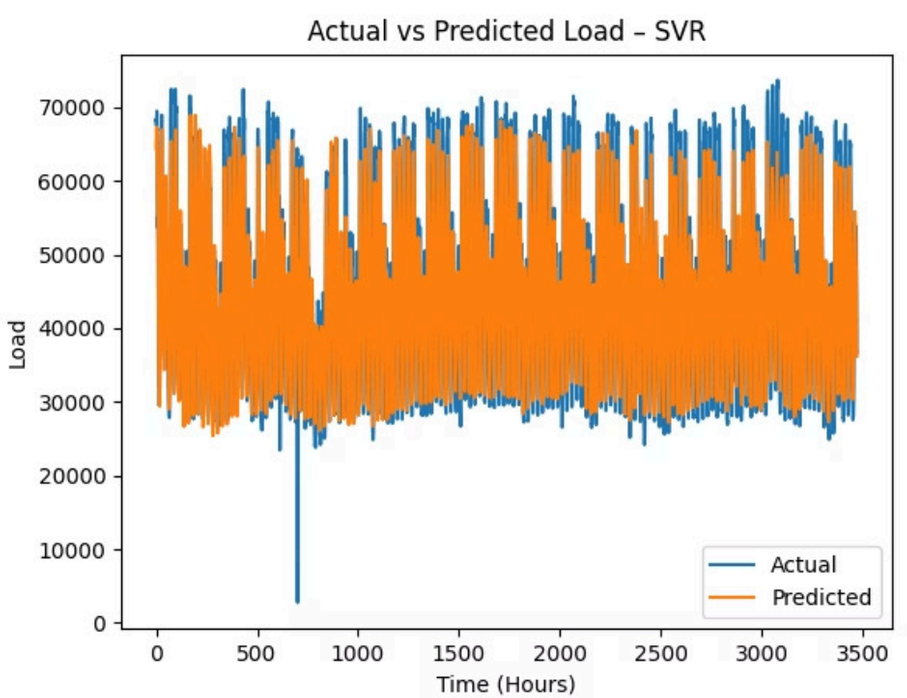
MAE: 3048

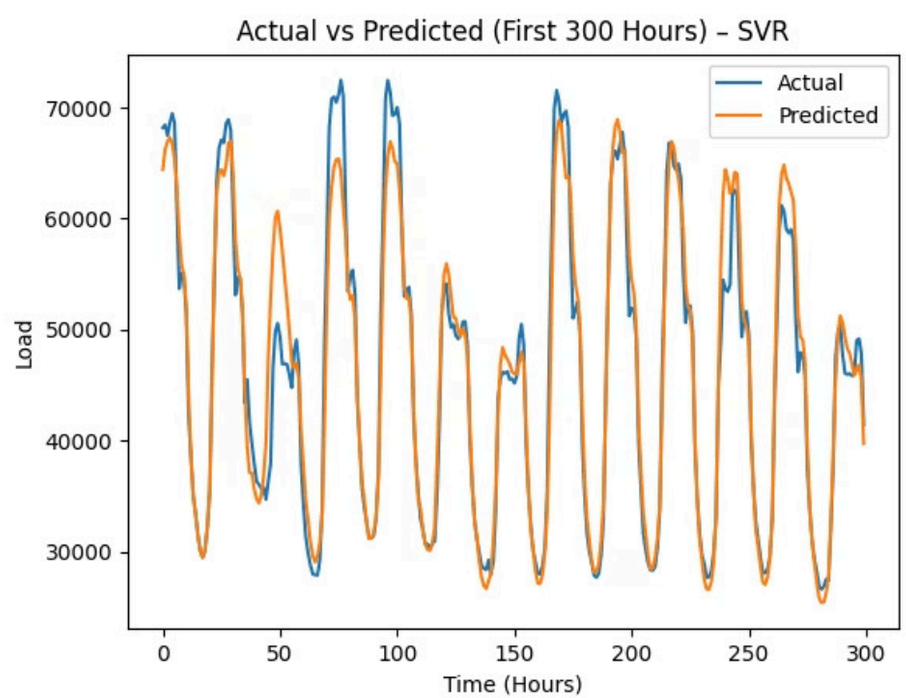
MAPE: 7.77%

sMAPE: 6.79%

R^2: 0.909

MBE: +530





Made with GAMMA

XGBoost

RMSE: 1927

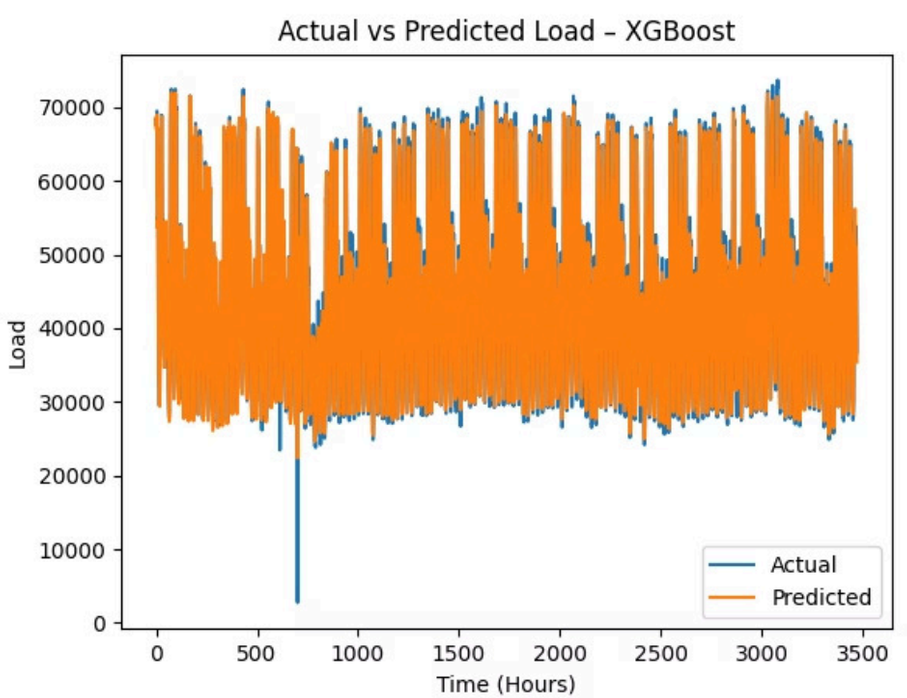
MAE: 1205

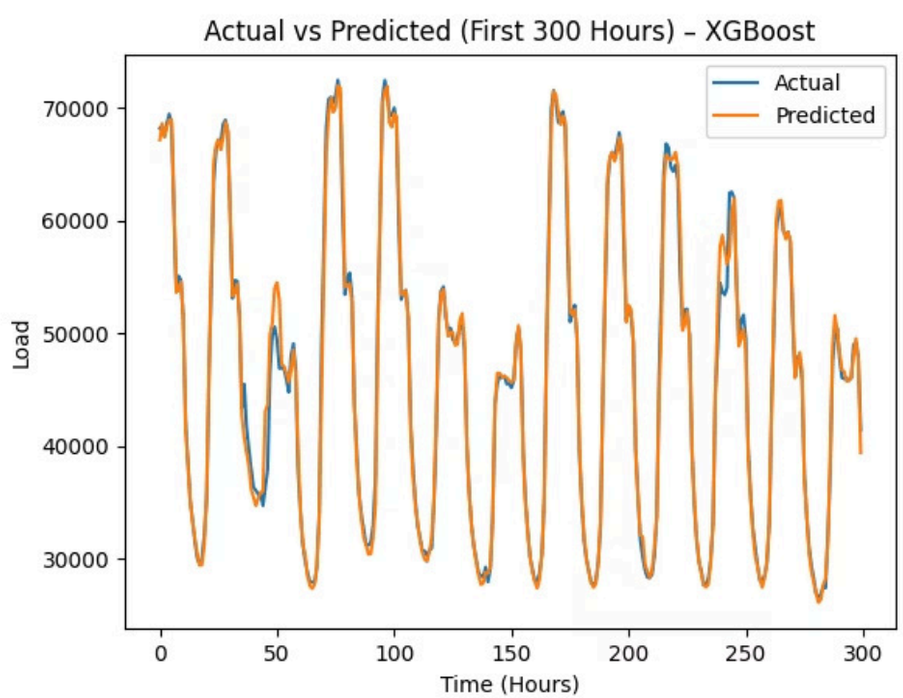
MAPE: 3.23%

sMAPE: 2.73%

R^2: 0.981

MBE: +334





Made with GAMMA

CatBoost

RMSE: 2262

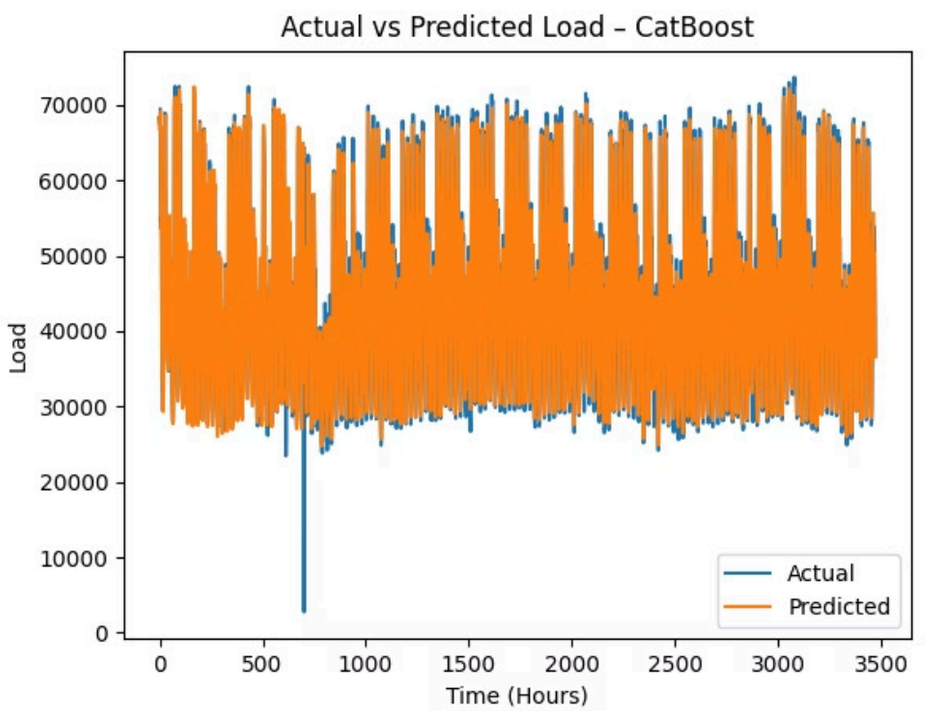
MAE: 1448

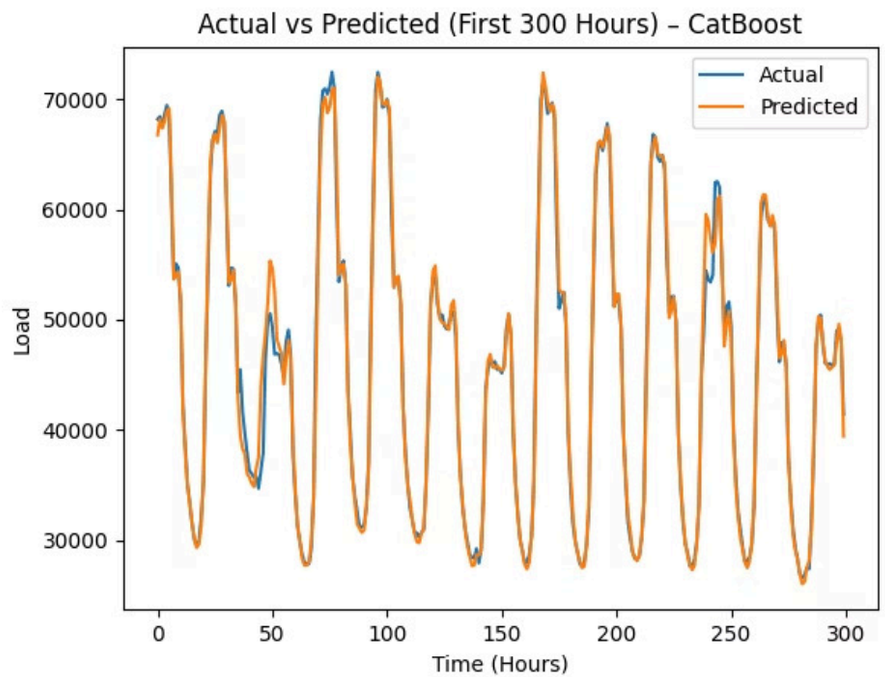
MAPE: 4.04%

sMAPE: 3.35%

R^2: 0.974

MBE: +284





Made with GAMMA

Observation

- XGBoost achieved highest accuracy with lowest RMSE (1927) and highest R^2 (0.981).
- Random Forest performed very close to XGBoost with low MAPE (3.02%).
- SVR showed weakest performance with highest RMSE (4212) and lowest R^2 (0.909).
- All tree-based models (RF, XGBoost, CatBoost) achieved $R^2 > 0.97$, indicating strong prediction capability.
- Overall, XGBoost is the best model for short-term load forecasting based on error metrics and accuracy.

Made with GAMMA

References

[1] L. Zhang & D. Jánošík, “Enhanced short-term load forecasting with hybrid machine learning models: CatBoost and XGBoost approaches,” Expert Systems with Applications, vol. 241, 2024.

[2] A. Lahouar & J. Ben Hadj Slama, “Day-ahead load forecast using random forest and expert input selection,” Energy Conversion and Management, vol. 103, pp. 1040–1051, 2015.

[3] Y. Chen et al., “Short-term electrical load forecasting using the Support Vector Regression (SVR) model to calculate the demand response baseline for office buildings,” Applied Energy, vol. 195, pp. 659–670, 2017.

[4] Q. Yang, Y. Lin, S. Kuang, et al., “A novel short-term load forecasting approach for data poor areas based on K-MIFS-XGBoost and transfer-learning,” Electric Power Systems Research, vol. 229, 110151, 2024.

[5] Saima Akhtar, Sulman Shahzad., “Short-Term Load Forecasting Models: A Review of Challenges, Progress, and the Road Ahead,” Energies, vol. 16, no. 10, article 4060, 2023.

Made with GAMMA

Thank you

Made with GAMMA